Stealtooth: Breaking Bluetooth Security Abusing Silent Automatic Pairing

Keiichiro Kimura*
Kobe University
Kobe, Japan
233t225t@gsuite.kobe-u.ac.jp

Hiroki Kuzuno
Kobe University
Kobe, Japan
kuzuno@port.kobe-u.ac.jp

Yoshiaki Shiraishi
Kobe University
Kobe, Japan
zenmei@port.kobe-u.ac.jp

Masakatu Morii
Kobe University
Kobe, Japan
mmorii@kobe-u.ac.jp

arXiv:2507.00847v2 [cs.CR] 3 Jul 2025

Abstract

Bluetooth is a pervasive wireless communication technology used by billions of devices for short-range connectivity. The security of Bluetooth relies on the pairing process, where devices establish shared long-term keys for secure communications. However, many commercial Bluetooth devices implement automatic pairing functions to improve user convenience, creating a previously unexplored attack surface.

We present **Stealtooth**, a novel attack that abuses unknown vulnerabilities in the automatic pairing functions in commercial Bluetooth devices to achieve completely silent device link key overwriting. The Stealtooth attack leverages the fact that Bluetooth audio devices automatically transition to pairing mode under specific conditions, enabling attackers to hijack pairing processes without user awareness or specialized tools. We also extend the attack into the **MitM Stealtooth** attack, combining automatic pairing abuse with power-saving mode techniques to enable man-in-the-middle attacks.

We evaluate the attacks against 10 commercial Bluetooth devices from major manufacturers, demonstrating widespread vulnerabilities across diverse device types and manufacturers. Our practical implementation requires only commodity hardware and open-source software, highlighting the low barrier to entry for attackers.

We propose defenses both device and protocol levels, including enhanced user notifications and standardized automatic pairing guidelines. Our findings reveal a critical tension between security and usability, showing that current automatic pairing implementations create systematic vulnerabilities. We responsibly disclosed our findings to affected vendors, with several already releasing patches.

xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
Conference'17, Washington, DC, USA
© 2025 xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
XXX XXXX 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/xxxxxxx/nnnnnnn.nnnnnnn

Keywords

Do, Not, Us, This, Code, Put, the, Correct, Terms, for, Your, Paper

ACM Reference Format:
Keiichiro Kimura, Hiroki Kuzuno, Yoshiaki Shiraishi, and Masakatu Morii. 2025. Stealtooth: Breaking Bluetooth Security Abusing Silent Automatic Pairing. In . xxxxxxxxxxxxxxxxxxxxxx , 13 pages. https://doi.org/xxxxxxx/nnnnnnn.nnnnnnn

1 Introduction

Bluetooth has become one of the most ubiquitous wireless communication technologies, with billions of devices relying on it for short-range connectivity across personal computing, IoT, peripheral, and wearable applications[23–25]. The widespread adoption of Bluetooth across diverse device types—from smartphones and laptops to headsets, keyboards, and industrial sensors—has made it a critical component of modern digital infrastructure[4, 12, 17, 18, 20]. As these devices increasingly handle sensitive data including audio communications, personal files, and control commands, the security of Bluetooth connections has become paramount.

The security architecture of Bluetooth relies fundamentally on the pairing process, where devices establish a shared long-term key (link key) that serves as the foundation for all subsequent secure communications[3, 31]. The pairing process, typically requiring user confirmation, creates a trusted relationship and secure sessions between devices that should prevent unauthorized access and impersonation attacks. Once paired, devices can automatically reconnect and establish secure sessions without repeated user intervention, enabling the seamless connectivity that users expect from Bluetooth technology.

However, the convenience comes with inherent security risks. The automatic reconnection functionality, while improving user experience, creates potential attack vectors that have not been thoroughly investigated by the security community. Many commercial Bluetooth devices, particularly audio peripherals like headsets and speakers, implement automatic pairing functions designed to simplify the user experience by reducing the need for manual intervention during connection establishment. The automatic pairing functions allow devices to transition automatically into pairing states under certain conditions, such as when attempting to reconnect to previously paired devices.

Prior work has extensively investigated various Bluetooth security vulnerabilities, revealing critical weaknesses in different

aspects of the protocol. Notable attacks such as BIAS (Bluetooth Impersonation AttackS) [5] and Blacktooth [1] have primarily focused on abusing authentication weaknesses, role switching vulnerabilities, and encryption key negotiation flaws in the core Bluetooth specification. The BIAS attack abuses unidirectional authentication procedures to enable device impersonation, while Blacktooth leverages the flexibility of Master/Slave role assignments to establish unauthorized connections. Additionally, attacks like KNOB [6] have demonstrated vulnerabilities in encryption key negotiation processes that can significantly weaken communication security.

Beyond Bluetooth-specific attacks, the broader domain of wireless communication security has seen significant research attention in recent years. Studies on wireless device pairing vulnerabilities have identified various attack vectors, including acoustic eavesdropping during pairing processes [13] and method confusion attacks that exploit differences in authentication methods [28, 30]. Research on WiFi impersonation detection [2] and radio frequency fingerprinting [22] has also contributed to our understanding of wireless communication security challenges.

However, despite this extensive body of work, prior attacks have not analyzed the security implications of automatic pairing functions that are increasingly common in consumer Bluetooth devices. This represents a significant gap in the current understanding of Bluetooth security, as automatic functionalities may introduce novel attack vectors that bypass traditional security assumptions about user involvement in the pairing process. Unlike specification-level vulnerabilities that affect all compliant devices, automatic pairing behaviors vary significantly across manufacturers and device types, creating a diverse and previously unexplored attack surface.

Our work addresses this gap by investigating automatic pairing vulnerabilities for the first time, revealing a critical but overlooked attack surface in modern Bluetooth implementations. In this paper, we present **Stealtooth**, a novel attack that abuses previously unexplored vulnerabilities in Bluetooth's automatic pairing functions. The Stealtooth attack achieves *completely silent* and *stealthy* link key overwriting by abusing the automatic pairing functions implemented in commercial Bluetooth devices. The Stealtooth attack leverages the fact that many Bluetooth audio devices automatically transition to pairing functionalities when they cannot establish connections with previously paired devices, creating an opportunity for attackers to hijack these pairing processes without any user awareness.

We demonstrate that the Stealtooth attack can be extended into a man-in-the-middle (MitM) attack, called the **MitM Stealtooth** attack, by combining it with existing techniques that abuse Bluetooth power-saving modes. The MitM Stealtooth attack enables attackers to position themselves between communicating devices and intercept, modify, or relay communications while maintaining the appearance of normal operation to the victims.

The implications of our findings are *significant* for the broader Bluetooth ecosystem. The Stealtooth attacks work against commercially available devices without requiring any specialized hardware or deep protocol knowledge, making them practical threats that could be deployed by adversaries with minimal technical skills. Furthermore, the silent nature of the attacks makes them particularly dangerous, as users have no indication that their devices have been compromised.

We summarize our main contributions as follows:

- We unveil and demonstrate a *novel* vulnerability in Bluetooth automatic pairing functions that enables completely silent link key overwriting and propose the **Stealtooth** attack, which abuses the new vulnerabilities to establish malicious Bluetooth sessions with victim devices. The Stealtooth attack requires *no* specialized hardware, protocol manipulation, or user interaction, making it highly practical and deployable by adversaries with minimal technical skills.
- We also present the **MitM Stealtooth** attack, which extends the Stealtooth attack into a MitM attack by combining automatic pairing mode abuse with existing power-saving mode techniques. The MitM Stealtooth attack enables attackers to intercept, modify, and relay communications between victims while maintaining operational transparency.
- We conduct a comprehensive evaluation of the Stealtooth attacks against 10 commercial Bluetooth devices from major manufacturers, demonstrating the widespread nature of automatic pairing vulnerabilities. Our evaluation covers diverse device types including headphones, earbuds, and speakers from Sony, Anker, Google, Xiaomi, and other manufacturers. We also propose practical defenses to mitigate the automatic pairing vulnerabilities and discuss the limitations of our current attack implementation.

Ethical Consideration and Responsible Disclosure: This work investigates unknown threats to widespread technologies and proposes defenses. All experiments were conducted in-house; no external devices were attacked. We responsibly disclosed our findings to the related manufacturers. Several manufacturers acknowledged our findings. Sony has already released patches for the vulnerable devices we reported, with an acknowledgment to us.

The remainder of this paper is organized as follows: Section 2 provides background on Bluetooth technology and connection establishment processes. Section 3 defines our threat model and attack assumptions. Section 4 presents the detailed design of both the Stealtooth and MitM Stealtooth attacks. Section 5 describes our implementation approach. Section 6 presents comprehensive evaluation results against real-world devices. Section 7 discusses defense mechanisms and limitations. Section 8 reviews related work, and Section 9 concludes the paper.

2 Background

2.1 Bluetooth

Bluetooth is a short-range wireless communication technology widely used by billions of personal computing, IoT, peripheral, and wearable devices for low-power communication. The technology operates in the 2.4GHz ISM band and enables devices to exchange commands and data such as keyboard/mouse inputs, audio, and files through secure communication channels.

The Bluetooth technology consists of two main variants: Bluetooth Basic Rate/Extended Data Rate (BR/EDR), also known as Bluetooth Classic, and Bluetooth Low Energy (BLE)[26]. This work focuses on Bluetooth BR/EDR, which we refer to simply as Bluetooth throughout this paper. Bluetooth uses frequency hopping

spread spectrum across 79 channels, each with a bandwidth of 1 MHz.

The Bluetooth system architecture comprises two main components: the Bluetooth Controller and the Bluetooth Host[11]. The Controller implements the physical and logical layers in the Bluetooth chip, while the Host implements the L2CAP layer and application-oriented protocols in the device operating system. These components communicate through the Host Controller Interface (HCI).

In Bluetooth networks, devices are organized in a piconet structure where one device acts as the Master and up to seven other devices serve as Slaves. The Master provides the reference clock signal to synchronize all devices in the network. Importantly, device roles are not fixed—any Bluetooth device can initiate a connection and become the Master, regardless of its functionality or previous role. For example, Bluetooth headsets, typically slave devices, can take master roles and proactively initiate connections with smartphones. Devices can also switch Master-Slave roles after establishing a piconet.

2.2 Bluetooth Pairing And Connection

Establishing communication between Bluetooth devices involves two distinct processes: pairing and connection establishment[16]. These processes implement multiple security mechanisms including encryption, authentication, and authorization to protect sensitive data transmission.

2.2.1 Pairing Process. Pairing is performed when two Bluetooth devices meet for the first time and need to establish a shared long-term key, known as the link key (K_L)[10]. The most secure and widespread pairing mechanism is Secure Simple Pairing (SSP), which uses Elliptic Curve Diffie-Hellman (ECDH) for key agreement[7, 26, 27, 32]. If both devices support Secure Connections, SSP is performed on the P-256 curve; otherwise, it uses the P-192 curve[5]. During pairing, user confirmation is typically required when a new device attempts to pair with the user's device to prevent unauthorized connections. The pairing process generates a link key that serves as the foundation for all future secure connections between the two devices. This key is stored on both devices and remains valid for subsequent connection attempts, eliminating the need to repeat the pairing process.

2.2.2 Connection Establishment. Once devices are paired and share a link key, they can establish multiple secure connections using different session keys derived from the long-term key and public parameters. However, the Bluetooth specification also supports temporary connections between unpaired devices for basic communication, service discovery, and legacy compatibility purposes.

The connection establishment process involves several phases:

Initial Connection Setup: The Master device first queries the Slave to obtain device name and features, then sends a Connection Request to initiate connection establishment. This process leverages the flexible role assignment in Bluetooth, where the device initiating the connection becomes the Master.

Authentication. : Bluetooth provides two authentication mechanisms depending on device capabilities. Legacy Authentication is used for Legacy Secure Connections and provides unilateral authentication where typically only the Master authenticates the Slave[3]. Secure Authentication is used for Secure Connections and provides mutual authentication where both devices must prove possession of the shared link key.

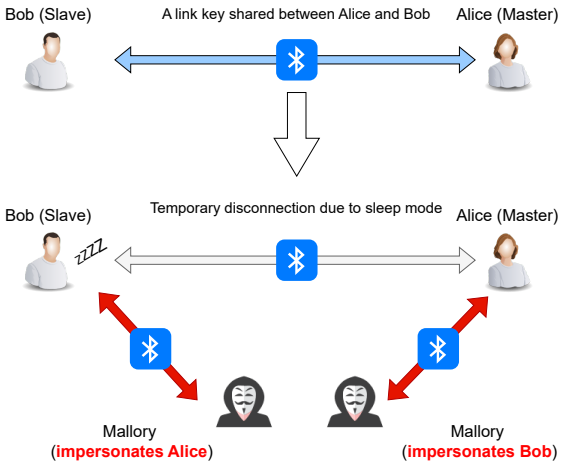


Figure 1: Threat model: Alice and Bob share a link key in advance. Mallory does not know the link key pre-shared between the victims. Mallory aims to establish Bluetooth sessions with Alice and Bob and maliciously control communications and operations between Alice and Bob.

Encryption Key Negotiation. : After successful authentication, devices negotiate an encryption key for the session[14]. The entropy of this encryption key ranges from 1 to 16 bytes according to the key length negotiation procedure. For Secure Connections, AES CCM encryption is used, while Legacy Secure Connections employ the E_0 stream cipher[1, 6].

Profile Management. : Bluetooth profiles define the protocols and functions required for specific device interactions[1]. Common profiles include the Advanced Audio Distribution Profile (A2DP) for audio communication, Human Interface Device Profile (HID) for keyboards and mice, and Phone Book Access Profile (PBAP) for contact access. Devices can implement multiple profiles simultaneously and advertise their available services using the Service Discovery Protocol (SDP).

3 Threat Model

In this section, we define our system and attacker models (Figure 1), as well as the notation we use in the rest of the paper.

3.1 System Model

We consider Alice and Bob (i.e., victims) who communicate securely via Bluetooth. Alice and Bob represent arbitrary Bluetooth devices and support arbitrary Bluetooth profiles (e.g., HID (Human Interface Device Profile), A2DP (Advanced Audio Distribution Profile), AVRCP (A/V Remote Control Profile)).

Without loss of generality, we assume Alice is the master and Bob is the slave. We assume the victims have previously paired and share a link key.

3.2 Attacker Model

We consider an attacker Mallory who aims to impersonate Alice, establish a secure connection with Bob, and use Bluetooth profiles to maliciously send data to Bob. Furthermore, Mallory also aims to impersonate Bob, establish a secure connection with Alice, and use the Bluetooth profile access permissions to control Alice's operations.

Mallory must be physically present within the Bluetooth range of the victims. Mallory does not know about the secure pairing process between the victims or their link keys. Mallory can capture unencrypted Bluetooth packets and recognize public information such as the victims' Bluetooth names and addresses.

3.3 Notations

In this paper, we denote the link key shared between Alice and Bob as LK_{AB}. Additionally, we denote the link key shared between Mallory and Alice as LK_{MA}, and the link key shared between Mallory and Bob as LK_{MB}.

4 The Stealtooth Attack

4.1 Attack Description

4.1.1 Strategy. The Stealtooth attack abuses a *new* vulnerability that exists in Bluetooth's pairing mode, allowing Mallory to establish a *completely stealthy* Bluetooth session with Bob. In this section, we explain the attack strategy of the Stealtooth attack.

The Stealtooth attack abuses a *new* improper authentication vulnerability present in Bluetooth audio devices (i.e., Bob). The sequence of the vulnerability is shown in Figure 2 and Figure 3. The vulnerability is such that when Mallory is impersonating a device paired with Bob (i.e., Alice), even if Bob is *not* explicitly in pairing mode and *without* requiring any operations from Bob's user, Bob will *actively* (or passively, depending on the device) connect and pair with Mallory. The attack strategy using the new vulnerability is as follows. First, Mallory changes her own Bluetooth address and adapter name to match Alice's settings. Next, Mallory sets the status of her Bluetooth adapter to `Discoverable` so that it can be detected by third-party devices[29]. After setting the adapter status, Mallory waits until Bob has been inactive for a certain period of time or is manually powered off. Subsequently, when Alice is powered off or her Bluetooth adapter status is set to off, and Bob is powered on again, Bob will *actively* (or passively, depending on the device) connect and pair with Mallory. At this time, during the pairing process, there are *no* notifications or permission requests to the users of Bob or Alice, thereby achieving a *completely stealth* attack.

4.1.2 Reproducibility. The Stealtooth attack is highly reproducible. To demonstrate that the attack is realistic, we describe a reproduction scenario. In the reproduction scenario, we consider a smartphone, Trent, which supports Bluetooth. Also, we assume that Alice is a laptop supporting Bluetooth, and Bob has already paired with both Alice and Trent. For example, Alice and Bob establish a Bluetooth session for a video conference. After the video conference ends, Alice and Bob turn off their power. Next, Trent attempts to connect with Bob to listen to music playing on the smartphone. However, if Mallory impersonating Alice exists within Bob's Bluetooth communication range, Bob improperly authenticates with Mallory before receiving the connection request from Trent. This reproduction scenario is the scenario in which we discovered the new vulnerability.

4.1.3 Impact. The Stealtooth attack has a severe impact on Bluetooth security and privacy. The Stealtooth attack can easily overwrite the link key required for Bluetooth communication and maliciously hijack Bluetooth sessions between victims. Furthermore, using the hijacked session, Mallory can decrypt data transmitted from victims or inject authorized messages to victims. Importantly, the attack has *high stealthiness* because pairing is executed *without* triggering any user operations or notifications, despite the fact that Mallory has *never* previously paired with Bob, meaning Mallory holds *no* information about Bob whatsoever. Additionally, the attack strategy used by the Stealtooth attack is threatening because it is *completely unaffected* by the presence or absence of SSP settings during Alice's or Bob's Bluetooth session establishment. Regarding the reproduction of the vulnerability used in the Stealtooth attack, it requires *neither* special privileges or tools, *nor* the implementation and execution of any specific code.

4.2 Attack Root Causes

The root cause of the Stealtooth attack lies in the automatic pairing mode functionality implemented in Bob, which transitions to pairing mode without requiring user intervention. Hereafter, this functionality will be referred to as "*automatic pairing mode*". Bob's automatic pairing mode is triggered when Bob attempts to reconnect with Alice but cannot confirm the connection. Mallory abuses this transition to automatic pairing mode to *stealthily* overwrite the link key previously shared between Alice and Bob, circumventing both user intervention and notifications. The Stealtooth attack represents the *first* attack that stealthily overwrites link keys by abusing the vulnerability in automatic pairing mode.

The link key overwrite scenario due to automatic pairing mode vulnerabilities can be divided into the following two patterns (***Pattern#1*** and ***#2***) based on the difference in connection initiators for overwriting the link key.

Pattern#1: Bob is the connection initiator. Figure 2 shows the vulnerability sequence of improper authentication due to automatic pairing mode when the connection initiator is Bob. We assume that Alice and Bob are pre-paired and communicate securely. Meanwhile, Mallory impersonates Alice and makes herself discoverable to other Bluetooth devices. We consider the case where Bob's Bluetooth session with Alice is disconnected, and Alice is also unavailable for connection. In this scenario, when Bob recovers and restores the Bluetooth session, Bob attempts to connect to Mallory impersonating Alice. Since Bob misidentifies Mallory as Alice, device authentication fails due to the link key mismatch between Bob and Mallory. However, *regardless* of the device authentication result, Bob *actively* executes pairing while still misidentifying Mallory as Alice through automatic pairing mode. The connection initiator

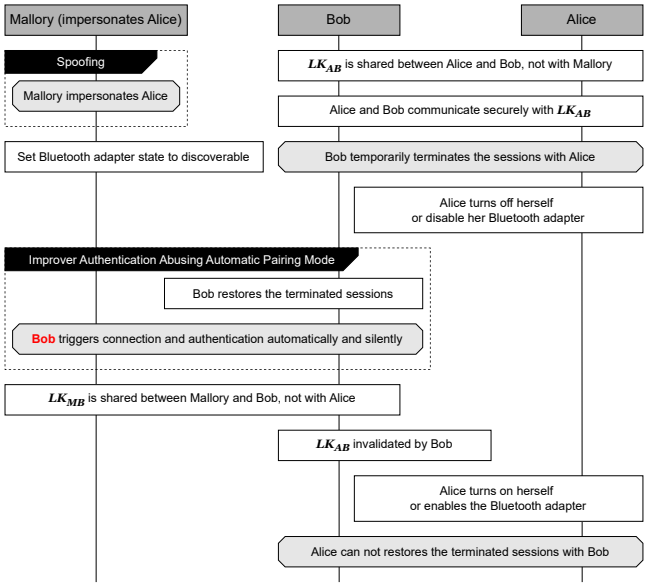


Figure 2: Improper Authentication Sequences (Bob is the connection initiator): We assume that Alice and Bob are paired, and Mallory impersonates Alice and is discoverable to other devices. When Bob's Bluetooth session with Alice is disconnected and Alice is also unavailable for connection, Bob, upon being powered on again, attempts to connect to Mallory impersonating Alice and *actively* executes pairing with Mallory through automatic pairing mode. To establish a malicious Bluetooth session with Bob, Mallory *only* needs to impersonate Alice and wait for Bob's recovery.

depends on the type of Bob device - the initiator may be either Bob or Mallory.

Pattern#2: Mallory is the connection initiator. Figure 3 shows the vulnerability sequence of improper authentication due to automatic pairing mode when the connection initiator is Mallory. Under the same assumptions as ***Pattern#1***, we consider the case where Bob attempts to restore the Bluetooth session again. Bob tries to connect to Mallory impersonating Alice. However, Mallory does not know the link key shared between Alice and Bob. Therefore, due to device authentication failure caused by the link key mismatch between Bob and Mallory, Bob disconnects the temporary connection with Mallory and transitions to automatic pairing mode. Meanwhile, through Bob's active connection attempt, Mallory recognizes Bob's Bluetooth address. At this point, when Mallory requests a connection to Bob while still impersonating Alice, Bob in automatic pairing mode pairs with Mallory regardless of the device authentication result and without requiring security-level verification during pairing.

The malicious establishment of Bluetooth sessions and link key overwriting through the above two patterns (***Pattern#1*** and ***#2***) both abuse the automatic pairing mode that devices secretly transition to. To the best of our knowledge, there have been *no* prior attacks against Bluetooth that focused on the vulnerabilities of automatic pairing mode.

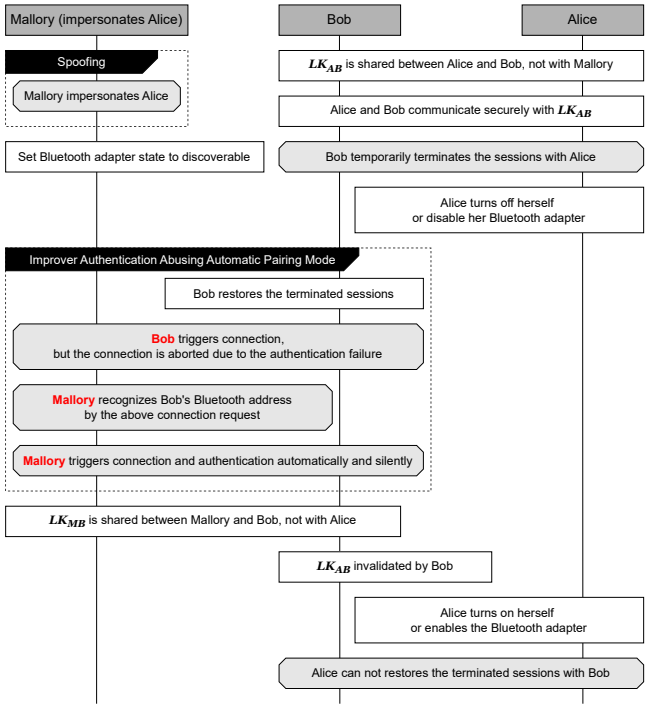


Figure 3: Improper Authentication Sequences (Mallory is the connection initiator): Considering the case where Bob is powered on again under the same assumptions as Figure 2, Bob attempts to connect to Mallory impersonating Alice, but disconnects from Mallory due to device authentication failure caused by link key mismatch. However, through Bob's connection attempt, Mallory recognizes Bob's Bluetooth address. At this point, when Mallory requests a connection to Bob while still impersonating Alice, Bob in automatic pairing mode pairs with Mallory.

4.3 The MitM Stealtooth Attacks

While the Stealtooth attack is completely stealthy, victims may still have a chance to notice they are under attack. For example, if an attacker launches the Stealtooth attack when the victim wants to use their Bluetooth headset, the victim will find they cannot connect to their headset and have an opportunity to discover the attack. To make the attack more concealed, we extend the Stealtooth attack to a MitM attack.

To extend the Stealtooth attack to a MitM attack, we employ the Breaktooth attack proposed by Kimura et al. in 2025, which hijacks device operations using Bluetooth power-saving mode. In this section, we first provide an overview of the Breaktooth attack. Then, we describe the attack strategy of the MitM Stealtooth attack, which combines the Stealtooth and Breaktooth attacks.

4.3.1 The Breaktooth Attack. In 2025, Kimura et al. proposed Breaktooth, a device hijacking attack that abuses Sleep mode, a Bluetooth power-saving mode[15]. The Breaktooth attack is the first attack to hijack victim device operations using Sleep mode. The Breaktooth attack abuses two vulnerabilities in Sleep mode: ***Vuln.#1***: the lack

of security notifications when Bluetooth sessions are disconnected, and ***Vuln.#2***: the vulnerability where the Master transitions to a state that accepts connection requests from Slaves after Bluetooth session disconnection. The attacker abuses ***Vuln.#1*** and ***#2*** to hijack Bluetooth sessions between victims without requiring any prior knowledge of link keys between victims, special privileges, or specialized tools. Furthermore, using the hijacked session as a starting point, the attacker gains complete control over the victim device operations.

The attack strategy of the Breaktooth attack consists of the following four steps (***Step#1-1*** to ***#1-4***):

- ***Step#1-1. Spoofing*** The attacker changes their Bluetooth name and address to impersonate the victim's Slave.
- ***Step#1-2. Session Hijacking*** The attacker, impersonating the victim's Slave from ***Step#1-1***, detects the temporary disconnection state of Bluetooth between victims due to Sleep mode, and at this moment sends a connection request to the Master as the victim's Slave to hijack the Bluetooth session between victims.
- ***Step#1-3. Link Key Hijacking*** After ***Step#1-2***, the attacker, while still impersonating the victim's Slave, sends a pairing request to the Master specifying low security-level authentication functions and generates a new link key between them.
- ***Step#1-4. Command Injection*** Exploiting the link key hijacked in ***Step#1-3***, the attacker uses sensitive profiles to control the victim Master's operations. For example, the attacker uses the Human Interface Device profile to inject malicious commands into the victim's Master.

The sequence of the Breaktooth attack is shown in Figure 4. To hijack sessions by abusing Sleep mode vulnerabilities, the attacker needs to remotely and secretly monitor the state of Bluetooth sessions between victims. The Breaktooth attack achieves this by maliciously using `l2ping`[9, 19, 21, 33]. Kimura et al. analyzed the echo request and response behavior of `l2ping`, enabling attackers to remotely and secretly recognize Bluetooth sessions between victims.

Kimura et al. developed a tool to demonstrate the Breaktooth attack and evaluated the attack against commercial Bluetooth keyboards, mice, and audio devices that support Sleep mode. The evaluation results showed that attackers could control the operations of Masters (e.g., laptops, smartphones). Additionally, Kimura et al. released the developed tool as open-source.

4.3.2 MitM Attacks Combining Stealtooth And Breaktooth.

This section introduces the MitM Stealtooth attack strategy. Figure 5 shows the details of the attack strategy. The strategy consists of the following five steps (***Step#2-1*** to ***#2-5***):

- ***Step#2-1. Spoofing*** Mallory changes her Bluetooth name and its address to impersonate Bob. The same applies when Mallory impersonates Alice.
- ***Step#2-2. Session Hijack*** Mallory, impersonating Bob, detects the temporary disconnection state of Bluetooth between Alice and Bob by the sleep mode, and at this moment sends a connection request to Alice as Bob, hijacking the Bluetooth session between Alice and Bob.

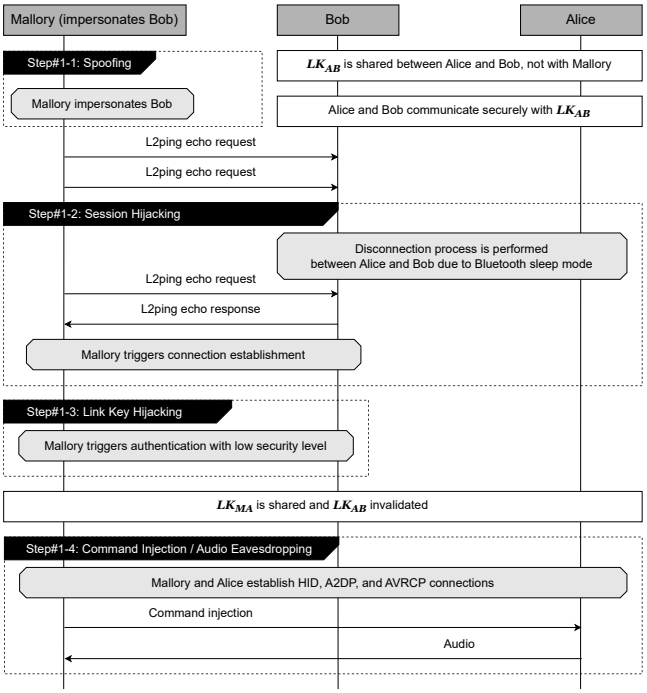


Figure 4: The Breaktooth attack sequences: The Breaktooth attack abuses a vulnerability where Alice transitions to a state that accepts reconnection requests from Bob after Bob enters Sleep mode as the attack starting point. Mallory detects Sleep mode from Alice's response behavior to malicious `l2ping` echo requests and abuses the Sleep mode vulnerability to hijack the Bluetooth session between Alice and Bob.

- ***Step#2-3. Link Key Hijack*** After the session hijack attack, Mallory sends a pairing request to Alice while impersonating Bob, generating a new link key between them (LK_{MA}). This invalidates LK_{AB}. Mallory does this by bypassing the PIN code authentication.
- ***Step#2-4. Improper Authentication*** Mallory, impersonating Alice, abuses the authentication vulnerability described in Section 4.1.1 and actively triggers Bob, who has returned from the sleep mode, to perform authentication, generating and sharing LK_{MB} between Mallory and Bob.
- ***Step#2-5. MitM Attacks*** Up to the improper authentication step, Mallory shares the link key LK_{MA} with Alice and the link key LK_{MB} with Bob. Therefore, Mallory becomes a MitM, capable of intercepting and manipulating the communication between Alice and Bob. For example, Mallory, impersonating Bob, establishes a connection with Alice using both the A2DP and AVRCP profiles, while Mallory, impersonating Alice, establishes a connection with Bob using the A2DP profile. In this scenario, Mallory can secretly eavesdrop on the audio data transmitted between Alice and Bob.

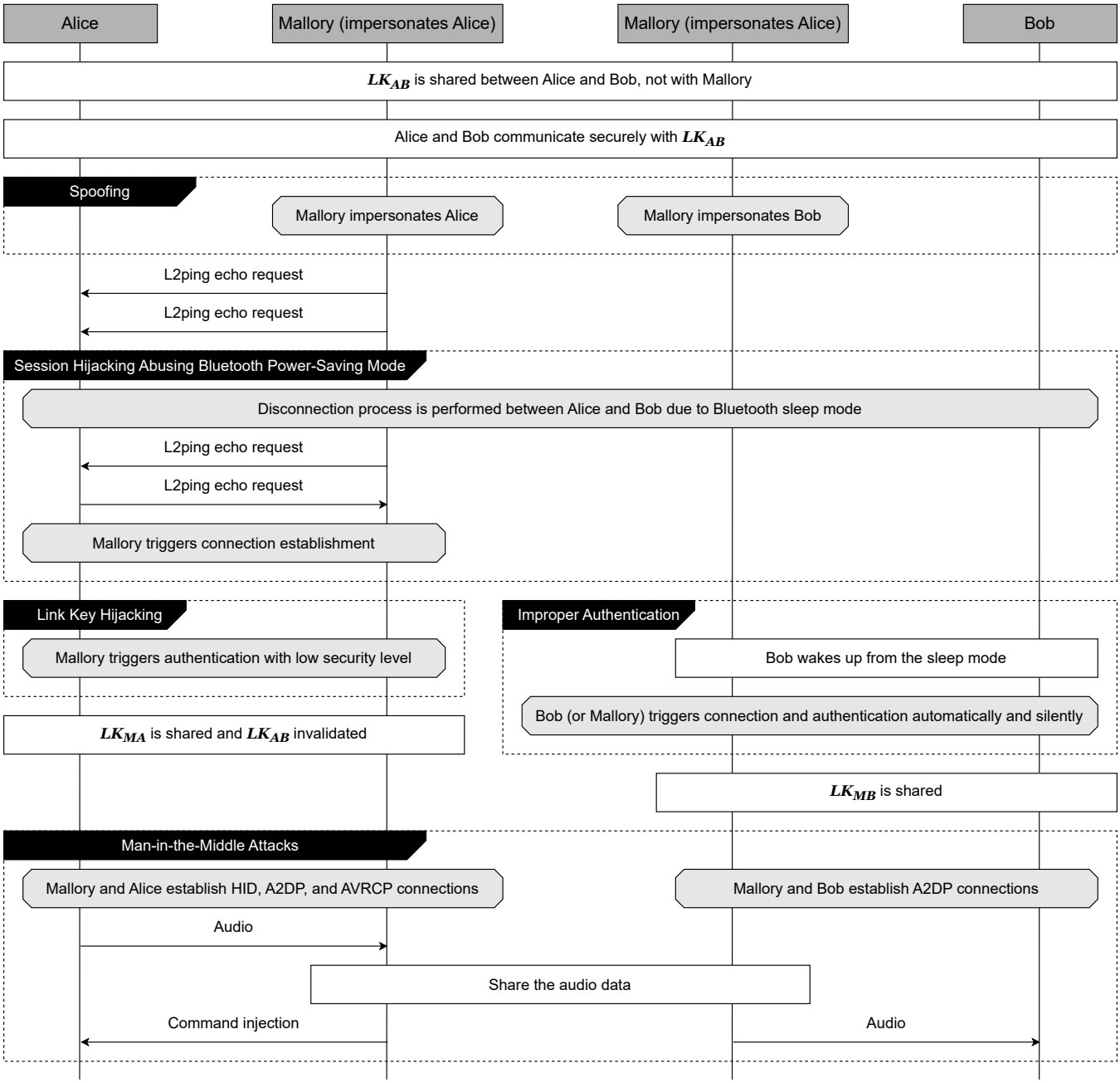


Figure 5: The MitM Stealtooth attack strategy: Alice and Bob have already paired and share LK_{AB}. If Bob remains inactive for a certain period, Bob terminates the session between Alice and Bob. After the termination, Mallory triggers connection establishment and low-level authentication. After the authentication, Mallory and Alice share LK_{MA}. On the other hand, when Bob restores the session, Mallory abuses the authentication vulnerability to pair with Bob, and shares LK_{MB} with Bob. Finally, Mallory stealthily proxies the communication between Alice and Bob as a MitM abusing LK_{MA} and LK_{MB}.

5 Implementation

As described in Section 4.1.3, the execution of the Stealtooth attack requires no specific attack code whatsoever. Therefore, this chapter focuses on the implementation of the MitM Stealtooth attack.

As described in Section 4.3.2, among the attack strategies of the MitM attack, ***Step#2-1*** to ***Step#2-3*** are identical to the Breaktooth attack. Therefore, we execute them using the open-source tool of the Breaktooth attack. Additionally, ***Step#2-4*** can be executed without requiring implementation. Therefore, this chapter describes the

implementation details for demonstrating the man-in-the-middle attack in ***Step#2-5***.

We implement the following two systems to intercept and relay audio data transmitted and received using A2DP between victim Bluetooth devices.

A2DP Sender: A2DP Sender operates on Mallory impersonating Bob. A2DP Sender forwards audio data received from Alice to the other Mallory impersonating Alice.

A2DP Receiver: A2DP Receiver operates on Mallory impersonating Alice. A2DP Receiver sends audio data forwarded from A2DP Sender to Bob.

A2DP Sender Implementation. A2DP Sender intercepts A2DP streams transmitted from Alice and forwards them to A2DP Receiver via TCP socket communication. For obtaining A2DP streams, we utilize the Bluetooth module of PulseAudio server. PulseAudio server cooperates with the A2DP stack provided by BlueZ to acquire Bluetooth audio data as PCM data.

To ensure session stability, we implement an independent thread that monitors TCP socket status and attempts reconnection with exponential backoff when disconnection is detected. We also manage captured PCM data with queue buffers to mitigate the effects of network delays.

A2DP Receiver Implementation. A2DP Receiver sends audio data forwarded from A2DP Sender to Bob using A2DP. This system operates as a TCP socket server and utilizes the BlueZ stack through PulseAudio server for audio stream transmission. Using PulseAudio enables retransmission of intercepted audio data as A2DP-compliant audio streams. Additionally, by leveraging the audio processing stack of PulseAudio and BlueZ, we implement conversion from PCM data to audio codecs used in A2DP (e.g., SBC) and stable audio stream transmission.

Our implementation assumes the BlueZ stack on the Linux kernel. We adopted BlueZ for attack demonstration because it is the most widely adopted open-source Bluetooth protocol stack.

6 Evaluation

In this section, we describe devices used for the Stealtooth attack evaluation and attack scenarios to evaluate the attack. We also present our evaluation setup and results.

6.1 Attack Devices

In the Stealtooth attack evaluation, we use a Raspberry Pi 4 Model B device as Mallory, as shown in Table 1. In the MitM Stealtooth attack evaluation, we use two Raspberry Pi shown in Table 1.

The Raspberry Pi runs on Raspberry Pi OS (11 Bullseye) with Linux OS kernel version 6.1. This operating system comes with BlueZ 5.55 pre-installed. We employ `hciconfig` for HCI configuration commands and `hcitool` for Bluetooth device scanning and enumeration operations[8]. Both commands are included in the BlueZ Bluetooth stack and are readily available upon OS installation. Additionally, Bluetooth adapter support is built into the Raspberry Pi by default. Consequently, the hardware/software cost of the Stealtooth attack is *low*.

Table 1: Specifications of devices used as Mallory in the Stealtooth attack: In case of the MitM Stealtooth attack evaluation, two devices are required as Mallory - one impersonating Alice and one impersonating Bob, but both use devices with the same specifications.

Device Model	Raspberry Pi 4 Model B
Operating System	Raspberry Pi OS
System	32bit
Debian Version	11 Bullseye
Kernel Version	6.1
BlueZ Version	5.55
Bluetooth Version	5.0

Table 2: Specifications of device used as Alice in the Breaktooth attack

Manufacturer	Model	Operating System	Bluetooth Version
Microsoft	Surface Laptop 4	Windows 11	5.1

Additionally, we use one device as Alice, as shown in Table 2. For Bob, we use 10 commercial Bluetooth headsets as shown in Table 3.

6.2 Attack Scenarios

To evaluate the effectiveness of the MitM Stealtooth attack as a man-in-the-middle attack, we define the following three attack scenarios: ***AS#1, #2, #3***.

***AS#1. Interception of communication*:** Mallory attempts to eavesdrop on audio data transmitted from Alice to Bob.

***AS#2. Tampering with communication*:** Mallory evaluates whether it is possible to send different audio data to Bob instead of the audio data received from Alice.

***AS#3. Proxying communication*:** Mallory evaluates whether audio data received from Alice can be seamlessly forwarded to Bob.

6.3 Setup

To evaluate the Stealtooth attack, we first pair the victims Alice and Bob and establish a Bluetooth session. Next, the attacker Mallory sets one device's Bluetooth name and address to match those of Alice.

Figure 6 shows the evaluation model of the MitM Stealtooth attack. For the evaluation of the MitM attack, after pairing the victims Alice and Bob and establishing a Bluetooth session, we keep Bob in an inactive state for a certain period until Bob enters idle mode. The attacker Mallory sets one device's Bluetooth name and address to match those of Bob, and the other to match those of Alice. We then install the attack tool "Breaktooth" on both Mallory devices. The Mallory impersonating Alice remotely and secretly monitors the Bluetooth session state between Alice and Bob until Bob transitions to idle state. The Bluetooth session state can be monitored using Breaktooth's functionality. Additionally, to transmit audio data intercepted from Alice to the other Mallory, we launch the A2DP Sender implemented in Section 5. Meanwhile, the Mallory impersonating Bob sets its Bluetooth adapter's `Discoverable` parameter to `yes`, making the Bluetooth adapter detectable. Furthermore, to

Table 3: Specifications of devices used as Bob

Manufacturer	Model	Bluetooth Version	Chip Producer	Chip Model
Sony	WH-1000XM5	5.2	MediaTek	MT2822
Sony	WH-1000XM4	5.0	MediaTek	MT2811
Sony	WF-1000XM5	5.3	-	-
Sony	WF-1000XM4	5.2	MediaTek	MT2822S
Anker	Soundcore Space One	5.3	-	-
EDIFIER	W820NB	5.0	-	-
TOZO	NC2	5.2	-	-
Xaomi	Redmi Buds 6 Pro	5.3	-	-
Google	Pixel Buds Pro	5.0	-	-
BOSE	Bose QuietComfort Ultra Headphones	5.3	Qualcomm	QCC5181

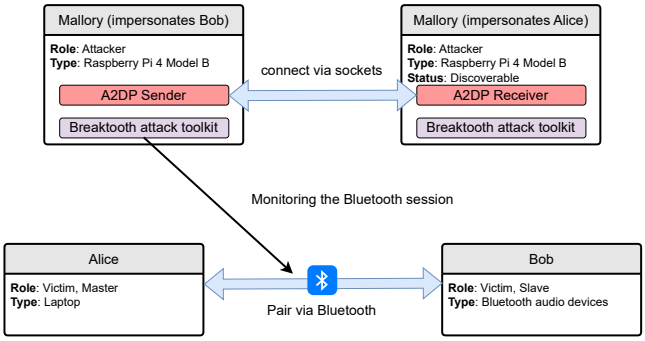


Figure 6: Evaluation model for the MitM Stealtooth attack: Alice and Bob are paired in advance. Meanwhile, both Raspberry Pis operating as Mallory have installed the Breaktooth attack toolkit. Additionally, A2DP Sender and A2DP Receiver implemented in Section 5 are launched, and communication is established between the two Mallory devices in advance

receive audio data transmitted from the A2DP Sender, we launch the A2DP Receiver implemented in Section 5.

6.4 Results

In this section, we present the evaluation results of our Stealtooth and MitM Stealtooth attacks against commercial Bluetooth devices.

6.4.1 Stealtooth Attack Results. The Stealtooth attack evaluation results are shown in Table 4. We successfully demonstrated the Stealtooth attack against 8 out of 10 tested devices, including Sony WH-1000XM5, WH-1000XM4, WF-1000XM5, WF-1000XM4, Anker Soundcore Space One, EDIFIER W820NB, and TOZO NC2. The attack was not successful against Google Pixel Buds Pro and BOSE QuietComfort Ultra Headphones.

The results reveal two distinct patterns based on the connection initiator. For Sony WH-1000XM5 and WF-1000XM5, Bob acts as the connection initiator (***Pattern#1***), actively attempting to reconnect with Alice and subsequently pairing with Mallory through automatic pairing mode. For the remaining vulnerable devices, Mallory serves as the connection initiator (***Pattern#2***), where Bob transitions to automatic pairing mode after failed authentication attempts, enabling Mallory to establish malicious connections.

Notably, the vulnerability appears to be implementation-specific rather than chip-specific, as devices with similar MediaTek chipsets (MT2822 and MT2822S) exhibit different behaviors. The successful attacks demonstrate complete link key overwriting without any user notification or intervention, confirming the silent nature of the automatic pairing mode abuse.

6.4.2 MitM Stealtooth Attack Results. The MitM Stealtooth attack results are also shown in Table 4. Among the 8 devices vulnerable to the Stealtooth attack, 4 devices are also vulnerabile to the MitM Stealtooth attack.

For attack scenario ***AS#1*** (interception of communication), the MitM Stealtooth attack successfully demonstrated the ability to intercept audio data transmitted from Alice to Bob. The attack captured and recorded the intercepted audio as WAV files, confirming the compromised confidentiality of Bluetooth communications.

Attack scenario ***AS#2*** (tampering with communication) was achieved against Sony WH-1000XM5 and WF-1000XM5 devices. The attack successfully demonstrated that while Mallory intercepts audio data from Alice, Mallory can simultaneously play pre-prepared audio data to Bob, effectively tampering with the communication content. For Sony WH-1000XM4 and WF-1000XM4, where Mallory serves as the connection initiator, AS#2 was achieved only after establishing the connection, disconnecting briefly, and reconnecting to properly grant profile access permissions.

Attack scenario ***AS#3*** (proxying communication) was partially achieved for the four tested devices. While Mallory impersonating Bob could transfer intercepted audio data from Alice via A2DP Sender, and Mallory impersonating Alice could receive the transferred audio data with A2DP Receiver, the received audio data could not be properly encoded into the appropriate audio codec when transmitting to Bob, preventing seamless audio playback.

6.4.3 Impact Assessment. The evaluation results demonstrate that abusing automatic pairing vulnerabilities enables attackers to become MitM adversaries in Bluetooth communications between victims. The successful achievement of attack scenarios ***AS#1*** and ***AS#2*** confirms that the MitM Stealtooth attack significantly compromises the confidentiality, integrity and availability of Bluetooth communications.

The widespread nature of these vulnerabilities across major manufacturers and different device types indicates that automatic pairing implementations represent systematic security weaknesses

Table 4: Stealtooth attack evaluation results

Model (Bob)	Stealtooth Attack		MitM Stealtooth Attack		
	Attack	Connection Initiator	***AS#1***	***AS#2***	***AS#3***
WH-1000XM5	✓	Bob	●	●	◐
WH-1000XM4	✓	Mallory	●	◐	◐
WF-1000XM5	✓	Bob	●	●	◐
WF-1000XM4	✓	Mallory	●	◐	◐
Soundcore Space One	✓	Mallory	-	-	-
W820NB	✓	Mallory	-	-	-
NC2	✓	Mallory	-	-	-
Redmi Buds 6 Pro	✓	Mallory	-	-	-
Pixel Buds Pro	✗	-	-	-	-
Bose QuietComfort Ultra Headphones	✗	-	-	-	-

✓ The Stealtooth attack is achieved.
✗ The Stealtooth attack is not achieved.
● Attack scenario is achieved.
◐ Attack scenario is partially achieved.
○ Attack scenario is not achieved.

rather than isolated flaws. The fact that 80% of tested devices were vulnerable to the Stealtooth attack, with 40% were vulnerable to the MitM attacks, underscores the critical impact of these findings on the broader Bluetooth ecosystem.

The results also reveal that the success of automatic pairing exploitation depends heavily on device-specific implementations, suggesting that manufacturers have varying approaches to automatic pairing that create *inconsistent* security behaviors across the market.

7 Discussion

In this section, we propose defenses against the Stealtooth attack. We also describe limitations of this work and future work.

7.1 Defense Against the Stealtooth Attack

To defend against the Stealtooth attack, we propose several defenses that can be implemented at different levels of the Bluetooth stack.

7.1.1 Device-Level Defenses. The most immediate defense against the Stealtooth attack lies in improving the transparency and control of automatic pairing mode. Bluetooth devices should notify users when they automatically transition to pairing mode and when new pairings are established. This notification mechanism would alert users to potential unauthorized pairing attempts and allow them to take appropriate action. However, such notifications must be carefully designed to balance security awareness with user experience, as excessive notifications could lead to alert fatigue.

Manufacturers should also consider implementing additional authentication layers for automatic pairing scenarios. The authentication layers could include device-specific challenges or cryptographic proofs that prevent unauthorized devices from successfully completing automatic pairing. Such mechanisms would need to be carefully designed to avoid breaking compatibility with existing paired devices while preventing unauthorized pairings.

7.1.2 Protocol-Level Improvements. At the protocol level, the Bluetooth specification should include mechanisms to validate the legitimacy of link key replacement during automatic pairing scenarios. This could involve cryptographic proofs or time-based validation tokens that ensure only legitimate devices can replace existing pairings. Such improvements would require careful consideration of backward compatibility and the diverse ecosystem of Bluetooth implementations.

Furthermore, the Bluetooth SIG should establish clear guidelines for automatic pairing implementations to ensure consistent security behaviors across different manufacturers and device types. The current lack of standardization in automatic pairing behavior creates inconsistent attack surfaces and makes it difficult to develop comprehensive defensive strategies.

7.2 Limitations and Future Work

7.2.1 Current Limitations. Our current implementation and evaluation have several limitations that constrain the scope of our findings. While we evaluated 10 commercial Bluetooth devices from major manufacturers, the automatic pairing behaviors vary significantly across device types and firmware versions. The diversity of implementations means that our findings may not generalize to all Bluetooth devices in the market. We need a more comprehensive evaluation across broader device categories and manufacturers to understand the vulnerability landscape.

The success of the Stealtooth attack also depends on specific timing conditions, such as when legitimate devices are powered off or unavailable. In real-world scenarios, these conditions may not always be controllable by attackers, limiting the practical applicability of the attack. The probabilistic nature of these timing dependencies means that attacks may require multiple attempts to succeed.

Our MitM Stealtooth attack implementation focuses primarily on A2DP audio interception, representing only a subset of the potential capabilities enabled by successful device impersonation. Extension

to other Bluetooth profiles and more sophisticated manipulation capabilities would require additional implementation work.

7.2.2 Future Work. Our work opens several important research directions that could significantly advance the understanding of Bluetooth security and automatic pairing vulnerabilities. In this section, we present two main future directions of this work.

Cross-Platform Vulnerability Analysis. While our evaluation focuses on commercial audio devices, automatic pairing modes exist across a much broader ecosystem of Bluetooth-enabled devices, including IoT sensors, medical devices, automotive systems, and industrial controllers. These diverse platforms may implement automatic pairing with different security assumptions and constraints, potentially exposing novel attack vectors. A systematic analysis of automatic pairing implementations across heterogeneous platforms could reveal fundamental design patterns that are inherently vulnerable, leading to more comprehensive mitigation strategies that address root causes rather than individual device vulnerabilities.

Large-Scale Empirical Security Assessment. The true scope of automatic pairing vulnerabilities in deployed devices remains unknown. Conducting large-scale empirical studies that systematically test thousands of devices across different manufacturers, firmware versions, and deployment contexts would provide crucial data for understanding the real-world impact of these vulnerabilities. Such studies could also reveal correlations between device characteristics and vulnerability patterns, informing risk assessment and prioritization of mitigation efforts.

8 Related Work

In this section, we describe prior MitM attacks against Bluetooth and present the differences between these prior attacks and the MitM Stealtooth attack.

8.1 Prior MitM Attacks Against Bluetooth

8.1.1 MitM Attack Abusing Unidirectionality of Authentication Procedure. The Bluetooth Impersonation AttackS (BIAS) proposed by Antonioli et al. is an attack that abuses the unidirectional vulnerability in Bluetooth's authentication procedure, allowing attackers to impersonate the victim's device and establish a connection[5]. Antonioli et al. evaluated the BIAS attack against 28 types of Bluetooth chips and demonstrated its effectiveness.

The BIAS attack can become a MitM attack when combined with the KNOB attack. Attackers can establish independent encrypted communication channels with both the sender and receiver devices using the BIAS attack, and then reduce the entropy of the session key using the KNOB attack to eavesdrop on or tamper with communications. The attack combining BIAS and KNOB attacks does not require any user operation or malicious applications. Furthermore, it can be executed on any Bluetooth device that complies with the standard Bluetooth protocol.

8.1.2 MitM Attack Using the Variability of Master/Slave Roles. The Blacktooth attack proposed by Ai et al. is the first attack that exploits the specification that Bluetooth Master/Slave roles are not fixed, allowing attackers to covertly complete connection and pairing with the victim's device[1]. The attacker establishes a connection by impersonating the legitimate device and bypasses authentication by exploiting the unidirectionality of legacy authentication. Additionally, they decrypt communication contents using the aforementioned KNOB attack to gain access permissions to highly confidential profiles and inject unauthorized commands to the victim. The Blacktooth attack can be executed without requiring any user operation or malicious applications. Ai et al. demonstrated the effectiveness of the Blacktooth attack against 21 types of Bluetooth devices.

8.1.3 MitM Attack Abusing Differences in Authentication Methods. The Method Confusion (MC) attack proposed by Tschirschnitz et al. is a MitM attack that exploits differences in authentication methods during Bluetooth pairing[28, 30]. In the MC attack, the attacker completes pairing as a MitM by using Numeric Comparison authentication with one device and Passkey Entry authentication with the other device. Tschirschnitz et al. evaluated the effectiveness of the MC attack with 40 users using devices such as smartwatches and smartphones, and successfully executed the MC attack against 37 users.

8.2 Comparison Between Prior MitM Attacks and the MitM Stealtooth Attack

The most significant difference between the prior attacks described in Section 2.3 and the conventional attacks and proposed attack lies in whether Bluetooth jamming between victims is necessary to initiate the attack.

The BIAS attack described in Section 8.1.1 requires forcibly disconnecting the Bluetooth session between victim devices to execute the attack. Therefore, Antonioli et al. assumed that in the attacker model for the BIAS attack, the attacker can jam the Bluetooth spectrum and has the ability to forcibly disconnect Bluetooth. However, they did not describe specific methods or feasibility of jamming.

The Blacktooth attack described in Section 8.1.2, like the BIAS attack, requires forcibly disconnecting the Bluetooth session between victim devices to execute the attack. However, similar to Antonioli et al., Ai et al. did not describe specific methods or feasibility of jamming.

The MC attack described in Section 8.1.3 also assumes jamming for its execution. The MC attack describes a specific method of selectively interfering with Bluetooth advertisement packets as "selective jamming". Unlike the BIAS attack and the Blacktooth attack, selective jamming does not jam the entire Bluetooth spectrum but only interferes with targeted packets. Selective jamming makes it possible to minimize the impact on other devices, making the attack less conspicuous.

The MitM Stealtooth attack easily achieves session hijacking including link key overwriting through the automatic pairing mode vulnerabilities we newly discovered. Furthermore, by combining with the existing Breaktooth attack, it abuses the vulnerabilities of Sleep mode that causes temporary Bluetooth disconnections without attacker intervention or victim operations, completely eliminating the jamming process from attack execution. The MitM Stealtooth attack provides a new attack vector against Bluetooth.

9 Conclusion

This paper unveils novel vulnerabilities of Bluetooth automatic pairing modes that enables completely silent device link key overwriting. We demonstrate how attackers can abuse the automatic pairing functions implemented in commercial Bluetooth devices to establish malicious connections without any user awareness or specialized equipment.

Our Stealtooth attack leverages the inherent behavior of commercial devices that automatically transition to pairing mode under specific conditions. We also extend the Stealtooth attack into a MitM attack, called MitM Stealtooth attacks, by combining the attack with existing power-saving mode techniques, enabling attackers to intercept, modify, and relay communications between victims.

We tested our attacks against 10 commercial Bluetooth devices from major manufacturers, including Sony, Anker, Google, and Xiaomi, and demonstrated the severe impact of the new vulnerabilities across various device chipsets, and vendors. Our practical implementation using only commodity hardware and open-source software highlights the real-world applicability of the attacks.

To address the critical impact of these vulnerabilities, we propose both device-level and protocol-level defenses. At the device level, we recommend enhanced user notification systems and stricter timeout mechanisms for automatic pairing. At the protocol level, we advocate for standardized automatic pairing guidelines and improved validation mechanisms for link key replacement scenarios.

Our findings reveal a critical tension between security and usability in wireless communication systems. While automatic pairing provides undeniable convenience benefits, current implementations inadequately consider security implications, creating systematic vulnerabilities rather than isolated implementation flaws.

Acknowledgments

This work is in part conducted under the "Research and development on new generation cryptography for secure wireless communication services" contract for the "Research and Development for Expansion of Radio Wave Resources (JPJ000254)", which is supported by the Ministry of Internal Affairs and Communications, Japan.

References

[1] Mingrui Ai, Kaiping Xue, Bo Luo, Lutong Chen, Nenghai Yu, Qibin Sun, and Feng Wu. 2022. Blacktooth: Breaking through the Defense of Bluetooth in Silence. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security* (Los Angeles, CA, USA) *(CCS '22)*. Association for Computing Machinery, New York, NY, USA, 55–68. doi:10.1145/3548606.3560668

[2] Muhamad Erza Aminanto, Rakyong Choi, Harry Chandra Tanuwidjaja, Paul D. Yoo, and Kwangjo Kim. 2018. Deep Abstraction and Weighted Feature Selection for Wi-Fi Impersonation Detection. *IEEE Transactions on Information Forensics and Security* 13, 3 (2018), 621–636. doi:10.1109/TIFS.2017.2762828

[3] Daniele Antonioli. 2023. BLUFFS: Bluetooth Forward and Future Secrecy Attacks and Defenses. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security* (Copenhagen, Denmark) *(CCS '23)*. Association for Computing Machinery, New York, NY, USA, 636–650. doi:10.1145/3576915.3623066

[4] Daniele Antonioli and Mathias Payer. 2022. On the Insecurity of Vehicles Against Protocol-Level Bluetooth Threats. In *2022 IEEE Security and Privacy Workshops (SPW)*. 353–362. doi:10.1109/SPW54247.2022.9833886

[5] Daniele Antonioli, Nils Ole Tippenhauer, and Kasper Rasmussen. 2020. BIAS: Bluetooth Impersonation AttackS. In *2020 IEEE Symposium on Security and Privacy (SP)*. 549–562. doi:10.1109/SP40000.2020.00093

[6] Daniele Antonioli, Nils Ole Tippenhauer, and Kasper B. Rasmussen. 2019. The KNOB is Broken: Exploiting Low Entropy in the Encryption Key Negotiation Of Bluetooth BR/EDR. In *28th USENIX Security Symposium (USENIX Security 19)*. USENIX Association, Santa Clara, CA, 1047–1061. https://www.usenix.org/conference/usenixsecurity19/presentation/antonioli

[7] Eli Biham and Lior Neumann. 2019. Breaking the Bluetooth Pairing – The Fixed Coordinate Invalid Curve Attack. In *Selected Areas in Cryptography – SAC 2019: 26th International Conference, Waterloo, ON, Canada, August 12–16, 2019, Revised Selected Papers* (Waterloo, ON, Canada). Springer-Verlag, Berlin, Heidelberg, 250–273. doi:10.1007/978-3-030-38471-5_11

[8] Eric Blancaflor, Harold Kobe Billo, John Michael Dignadice, Philip Domondon, Mico Ruiz Linco, and Christie Valero. 2025. Bluetooth Simulated Reconnaissance Attack Through the Use of HCITool: A Case Study. In *2nd International Conference on Cloud Computing and Computer Networks*, Lei Meng (Ed.). Springer Nature Switzerland, Cham, 133–143.

[9] Xijia Che, Yi He, Xuewei Feng, Kun Sun, Ke Xu, and Qi Li. 2024. BlueSWAT: A Lightweight State-Aware Security Framework for Bluetooth Low Energy. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security* (Salt Lake City, UT, USA) *(CCS '24)*. Association for Computing Machinery, New York, NY, USA, 2087–2101. doi:10.1145/3658644.3670397

[10] Marc Fischlin and Olga Sanina. 2024. Fake It till You Make It: Enhancing Security of Bluetooth Secure Connections via Deferrable Authentication. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security* (Salt Lake City, UT, USA) *(CCS '24)*. Association for Computing Machinery, New York, NY, USA, 4762–4776. doi:10.1145/3658644.3670360

[11] Matheus E. Garbelini, Vaibhav Bedi, Sudipta Chattopadhyay, Sumei Sun, and Ernest Kurniawan. 2022. BrakTooth: Causing Havoc on Bluetooth Link Manager via Directed Fuzzing. In *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, Boston, MA, 1025–1042. https://www.usenix.org/conference/usenixsecurity22/presentation/garbelini

[12] Rahul N. Gore, Himashri Kour, Mihit Gandhi, Deepaknath Tandur, and Anitha Varghese. 2019. Bluetooth based Sensor Monitoring in Industrial IoT Plants. In *2019 International Conference on Data Science and Communication (IconDSC)*. 1–6. doi:10.1109/IconDSC.2019.8816906

[13] Tzipora Halevi and Nitesh Saxena. 2013. Acoustic Eavesdropping Attacks on Constrained Wireless Device Pairing. *IEEE Transactions on Information Forensics and Security* 8, 3 (2013), 563–577. doi:10.1109/TIFS.2013.2247758

[14] Mohit Jangid, Yue Zhang, and Zhiqiang Lin. 2023. Extrapolating Formal Analysis to Uncover Attacks in Bluetooth Passkey Entry Pairing. In *2023, Network and Distributed System Security Symposium (NDSS)*. doi:10.14722/ndss.2023.23119

[15] Keiichiro Kimura, Hiroki Kuzuno, Yoshiaki Shiraishi, and Masakatu Morii. 2024. Breaktooth: Breaking Security and Privacy in Bluetooth Power-Saving Mode. Cryptology ePrint Archive, Paper 2024/900. https://eprint.iacr.org/2024/900

[16] Changseok Koh, Jonghoon Kwon, and Junbeom Hur. 2022. BLAP: Bluetooth Link Key Extraction and Page Blocking Attacks. In *2022 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. 227–238. doi:10.1109/DSN53405.2022.00033

[17] Grigorios Koulouras, Stylianos Katsoulis, and Fotios Zantalis. 2025. Evolution of Bluetooth Technology: BLE in the IoT Ecosystem. *Sensors* 25, 4 (2025). doi:10.3390/s25040996

[18] Elke Mackensen, Matthias Lai, and Thomas M. Wendt. 2012. Bluetooth Low Energy (BLE) based wireless sensors. In *SENSORS, 2012 IEEE*. 1–4. doi:10.1109/ICSENS.2012.6411303

[19] Linux man page. 2002-2024. l2ping(1). https://linux.die.net/man/1/l2ping. Accessed: 2025-06-01.

[20] Maja Pušnik, Mitja Galun, and Boštjan Šumak. 2020. Improved Bluetooth Low Energy Sensor Detection for Indoor Localization Services. *Sensors* 20, 8 (2020). doi:10.3390/s20082336

[21] Poonam Shelke, Saurav Gupta, and Sukumar Nandi. 2024. BlueDoS: A Novel Approach to Perform and Analyse DoS Attacks on Bluetooth Devices. In *Proceedings of the 21st International Conference on Security and Cryptography - Volume 1: SECRYPT*. INSTICC, SciTePress, 838–843. doi:10.5220/0012845700003767

[22] Guanxiong Shen, Junqing Zhang, Alan Marshall, and Joseph R. Cavallaro. 2022. Towards Scalable and Channel-Robust Radio Frequency Fingerprint Identification for LoRa. *IEEE Transactions on Information Forensics and Security* 17 (2022), 774–787. doi:10.1109/TIFS.2022.3152404

[23] Bluetooth SIG. 2023. 2023 Bluetooth® Market Update. https://www.bluetooth.com/2023-market-update/. Accessed: 2025-06-01.

[24] Bluetooth SIG. 2024. 2024 Bluetooth® Market Update. https://www.bluetooth.com/2024-market-update/. Accessed: 2025-06-01.

[25] Bluetooth SIG. 2025. 2025 Bluetooth® Market Update. https://www.bluetooth.com/2025-market-update/. Accessed: 2025-06-01.

[26] Da-Zhi Sun, Yi Mu, and Willy Susilo. 2018. Man-in-the-middle attacks on Secure Simple Pairing in Bluetooth standard V5.0 and its countermeasure. *Personal Ubiquitous Comput.* 22, 1 (Feb. 2018), 55–67. doi:10.1007/s00779-017-1081-6

[27] Da-Zhi Sun and Li Sun. 2019. On Secure Simple Pairing in Bluetooth Standard v5.0-Part I: Authenticated Link Key Security and Its Home Automation and Entertainment Applications. *Sensors* 19, 5 (2019). doi:10.3390/s19051158

[28] Maximilian Tschirschnitz, Ludwig Peuckert, Moritz Buhl, and Jens Grossklags. 2025. Rediscovering Method Confusion in Proposed Security Fixes for Bluetooth. In *2025, Network and Distributed System Security Symposium (NDSS)*. doi:10.14722/ndss.2025.240310

[29] Tyler Tucker, Hunter Searle, Kevin Butler, and Patrick Traynor. 2023. Blue's Clues: Practical Discovery of Non-Discoverable Bluetooth Devices. In *2023 IEEE Symposium on Security and Privacy (SP)*. 3098–3112. doi:10.1109/SP46215.2023.10179358

[30] Maximilian von Tschirschnitz, Ludwig Peuckert, Fabian Franzen, and Jens Grossklags. 2021. Method Confusion Attack on Bluetooth Pairing. In *2021 IEEE Symposium on Security and Privacy (SP)*. 1332–1347. doi:10.1109/SP40001.2021.00013

[31] Jianliang Wu, Yuhong Nan, Vireshwar Kumar, Dave (Jing) Tian, Antonio Bianchi, Mathias Payer, and Dongyan Xu. 2020. BLESA: Spoofing Attacks against Reconnections in Bluetooth Low Energy. In *14th USENIX Workshop on Offensive Technologies (WOOT 20)*. USENIX Association. https://www.usenix.org/conference/woot20/presentation/wu

[32] Jianliang Wu, Ruoyu Wu, Dongyan Xu, Dave Jing Tian, and Antonio Bianchi. 2022. Formal Model-Driven Discovery of Bluetooth Protocol Design Vulnerabilities. In *2022 IEEE Symposium on Security and Privacy (SP)*. 2285–2303. doi:10.1109/SP46214.2022.9833777

[33] Tuğrul Yüksel, Ömer Aydın, and Gökhan Dalkılıç. 2022. Performing DoS Attacks on Bluetooth Devices Paired with Google Home Mini. *SSRN Electronic Journal* 18 (01 2022), 53–58. doi:10.2139/ssrn.4171322